MODULE 07 · DOMAIN 1 - PLAN AND MANAGE AN AZURE AI SOLUTION

Securing AI systems

Keyless authentication, managed identity, key rotation, role policies, private endpoints, VNet injection and encryption.

DOMAIN 1 - Plan and manage WEIGHT 25-30% MODULE 07 of 39

admin@sthithapragna.com

Identity decides who may act. **Network** decides who can reach. A private endpoint grants no permission, and a role makes nothing private.

Microsoft Entra ID or API keys?

MICROSOFT ENTRA ID

Tokens, per identity

- Short-lived tokens, no stored secret
- Conditional access and MFA
- Least-privilege roles, per-principal audit
- Managed identities for Azure workloads

API KEYS

One shared secret

- Static, and never expire on their own
- Full access to the resource
- Cannot be scoped to users or actions
- Hard to audit: no user identity

THE RULE

Microsoft: use Entra ID for production. Keys are for prototyping and isolated test environments.

Three kinds of identity

IDENTITY	REPRESENTS	USE IT FOR
User principal	A person	Developers and portal users
Service principal	An application or automation	Workloads outside Azure, pipelines
Managed identity	An Azure resource	Code running in Azure: no secret to manage

THE SIGNAL

If the code runs on an Azure service, the answer is almost always a managed identity, not a service principal with a secret.

Moving an application off API keys

01
Identity
Enable a managed identity where the code runs.

02
Role
Assign a data plane role, such as Foundry User.

03
Code
Use a token credential instead of a key.

04
Confirm
Check every caller now uses tokens.

05
Disable
Set disableLocalAuth to true.

06
Enforce
Azure Policy: key access disabled.

ON THE EXAM

Order matters. Disable keys only after every caller has moved, or you cause the outage you were preventing.

Credentials in application code

- **DefaultAzureCredential** tries a sequence of sign-in methods, so one line works on a laptop and in Azure.
- **ChainedTokenCredential** lets you allow only specific methods, such as managed identity in production.
- **Never put keys in source code** or plain configuration committed to a repository.
- If a key truly cannot be avoided, store it in **Key Vault** and read it using a managed identity.

> ON THE EXAM
>
> Moving a key into Key Vault is better than leaving it in config, but it is still a key: full access, no user identity.

Rotating keys without an outage

- Every resource has **two keys**, KEY1 and KEY2, so rotation needs no downtime.
- **1.** Move every caller to one key, say KEY1. **2.** Regenerate KEY2.
- **3.** Move every caller to the new KEY2. **4.** Regenerate KEY1.
- A regenerated key **stops working immediately**. Callers still using it get 401 errors.
- Foundry Tools need a **custom subdomain** before they can use Entra ID instead of keys.

> ON THE EXAM
>
> Keys never expire on their own. Rotation is a process you must schedule and own.

There is no grace period. **Move the callers first**, then regenerate the key they have stopped using.

Managed identity to other Azure services

- Foundry reaches Storage and AI Search as **its own managed identity**, which needs roles there.
- Example: the Foundry resource's identity needs **Storage Blob Data Reader** to read a new blob container.
- Agent tools authenticate as the **agent identity**, which needs roles on the downstream service.
- Connected resources keep their own access control. Foundry roles grant nothing inside them.

> THE SIGNAL
>
> 'Foundry works, but cannot read the storage account' is a role missing on the storage account, not on Foundry.

Least privilege, persona by persona

PERSONA	ROLE AND SCOPE
App or user that only calls an agent	Foundry Agent Consumer, on the project or one agent
Developer building agents	Foundry User on the project, plus Reader on the resource
Pipeline that provisions and evaluates	Contributor on the resource group, plus Foundry User
Platform administrator	Foundry Account Owner on the resource

ON THE EXAM

Do not use Cognitive Services roles or the Azure AI Developer role for Foundry projects. They target other resource types.

Three places to isolate traffic

DIRECTION	WHAT IT PROTECTS	HOW
Inbound to Foundry	Who can reach the resource	Public network access flag and private endpoints
Outbound from Foundry	Calls to Storage, Key Vault and others	Private Link to each service
Outbound from agents	Tool calls to your data and APIs	VNet injection into your subnet

ON THE EXAM

Inbound isolation alone does not stop an agent calling out to the internet. Outbound is configured separately.

Private endpoints and DNS

- Public network access: **Disabled** with a private endpoint, **selected networks**, or all networks.
- Apps keep the **same endpoint name**. DNS returns the private IP inside the VNet, public outside.
- That needs a **privatelink private DNS zone** linked to the VNet, or custom DNS forwarding to Azure DNS.
- The private endpoint must be **approved**, and in the same region and subscription as the VNet.
- Reach a private resource from on-premises with **VPN** or **ExpressRoute**, or use a Bastion jump box.

> THE SIGNAL
>
> 'nslookup returns a public IP from inside the VNet' is always a DNS problem, never a role problem.

Networking surprises

- Removing private endpoints does **not** make a resource public. Enabling public access does not remove them.
- Private endpoints to your Storage, AI Search and Cosmos DB are **not** created for you.
- Outbound VNet injection **cannot be added later**. You must redeploy Foundry.
- The agent subnet must be delegated to **Microsoft.App/environments**, sized /27 or larger.
- Trusted services, such as AI Search, can reach a restricted Foundry using managed identity.

> ON THE EXAM
>
> Decide on outbound isolation before the first deployment. Retrofitting it means rebuilding.

Where agent tool traffic goes

TRAFFIC PATH	TOOLS
Through your subnet	Private MCP servers, OpenAPI, Azure Functions, agent-to-agent
Through a private endpoint	Azure AI Search, file search
Microsoft backbone network	Code interpreter, function calling
Public internet	Web search, Bing grounding, SharePoint grounding
Not yet supported	Browser automation, computer use, image generation, Logic Apps

WATCH FOR THIS

Public-endpoint tools still work in an isolated setup. If policy forbids public traffic, block them with Azure Policy.

Customer-managed keys

- By default, data at rest is encrypted with **Microsoft-managed keys**. No code changes needed.
- **Customer-managed keys** need Key Vault in the **same region** as the Foundry resource.
- Key Vault must have **soft delete and purge protection** enabled.
- The identity needs a key role, such as **Key Vault Crypto User**.
- Own keys for agent data also means **standard agent setup**, with your own stores.

> ON THE EXAM
>
> Encryption keys and API keys are unrelated. One protects data at rest; the other authenticates callers.

Security symptoms and their causes

SYMPTOM	MOST LIKELY CAUSE
401 errors right after a key rotation	Callers still using the key that was regenerated
403 errors, network path confirmed	Missing role on the Foundry resource or project
Public IP returned inside the VNet	Private DNS zone missing, unlinked, or not forwarded
Private endpoint stuck in Pending	Needs approval by the resource owner
Foundry cannot read a storage account	Foundry's identity lacks a role on that storage account

THE TIEBREAKER

401 means the credential failed. 403 means the credential worked but lacked permission.

> A web app on Azure App Service calls a Foundry project using an API key stored in its app settings.
>
> An audit requires no stored secrets, an audit trail showing which workload made each call, and the ability to ban key access across the organisation later.

What should the team do?

A. Move the API key into Key Vault and rotate it every 30 days.

B. Use a service principal, with its client secret stored in app settings.

C. Keep the key, and restrict Foundry's network to the app's outbound IPs.

D. Give the app a managed identity and a Foundry role, then disable keys.

◆ PAUSE AND DECIDE

What should the team do?

A. Key in Key Vault, rotated.
Better storage, but still a shared key: no per-workload audit, and keys can't be banned.

B. Service principal with a secret.
Adds identity, but a client secret in app settings is still a stored secret.

C. Keep the key, restrict network.
A network control. It does nothing for secrets or auditing.

D. Managed identity, role, disable keys.
No secret, calls attributed to the app's identity, and key access can then be disabled by policy.

THE SIGNAL

Three requirements, and only one option meets all three. Code on an Azure service points to managed identity.

> During quarterly key rotation, an administrator regenerated KEY1 on a Friday afternoon. Both production applications were using KEY1.
>
> Every request failed with 401 errors for an hour, until the applications were updated with the new key.

Which procedure would have avoided the outage?

A. Regenerate both keys together outside business hours, then update the apps.

B. Move both apps to KEY2, regenerate KEY1, and repeat later for KEY2.

C. Regenerate KEY1, then wait for the grace period before updating the apps.

D. Create a new Foundry resource each quarter, and move the apps across to it.

◆ PAUSE AND DECIDE

Which procedure would have avoided the outage?

A Regenerate both, out of hours.

Still an outage, just a quieter one. Nothing valid remains while apps are updated.

B Move apps to KEY2, then regenerate KEY1.

Callers always hold a valid key. This is the two-key rotation Microsoft documents.

C Wait for a grace period.

There is no grace period. The old key stops working immediately.

D New resource every quarter.

Needless disruption: new endpoints, deployments and quota, for a problem two keys solve.

THE TIEBREAKER

Two keys exist so that one is always valid. Any procedure that invalidates the key in use is wrong.

> A company adds a private endpoint to its Foundry resource and disables public network access. Its virtual networks use the company's own custom DNS servers.
>
> Applications on VMs in the network now fail to connect. From one VM, nslookup of the Foundry endpoint returns a public IP address.

What should the team do?

A Re-enable public network access for the company's selected networks.

B Assign the Foundry User role to each VM's managed identity.

C Have the custom DNS servers forward privatelink queries to Azure DNS.

D Recreate the private endpoint in a different region with more capacity.

◆ PAUSE AND DECIDE

What should the team do?

A. Re-enable public access.
Restores traffic by undoing the isolation the company just configured.

B. Assign Foundry User to the VMs.
A permission fix for a name resolution problem. Requests never reach the endpoint.

C. Forward privatelink queries to Azure DNS.
Custom DNS must resolve the privatelink name to the private endpoint's IP.

D. Recreate in another region.
A private endpoint must be in the VNet's region, and region isn't the fault.

THE SIGNAL

A public IP from inside the VNet is the fingerprint of broken private DNS. Custom DNS servers make it likely.

A bank runs agents in a network-isolated Foundry, with VNet injection. Policy states that no agent traffic may cross the public internet.

One agent uses a private AI Search index, a private MCP server inside the VNet, and the web search tool. Compliance has flagged the agent.

What should the team do?

A Remove web search, and block public-endpoint tools with Azure Policy.

B Create a private endpoint for the web search tool inside the agent subnet.

C Replace the private MCP server with a public one outside the VNet.

D Enable public network access on AI Search so the agent can reach it.

◆ PAUSE AND DECIDE

What should the team do?

A Remove web search, block with policy.

Web search always uses a public endpoint. Policy stops it being added again.

B Private endpoint for web search.

There is none. Web search reaches the public web by design.

C Make the MCP server public.

Moves a compliant tool onto the public internet. The opposite of the policy.

D Open AI Search to the public.

AI Search already works privately. This creates a new violation.

THE TIEBREAKER

Isolation does not stop public-endpoint tools working. Knowing which tools go public is the whole question.

Security in five lines

- **Identity decides who may act; network decides who can reach.** Fix the right axis.
- **Managed identity plus a role**, then disable keys, then enforce with Azure Policy.
- **Regenerated keys die instantly.** Move callers first.
- **Private DNS makes private endpoints work.** Outbound isolation is separate, and planned up front.
- **401 is the credential, 403 is the permission.** Web search stays public.

> ON THE EXAM
>
> Next, module 08: monitoring models, safety events, grounding quality and search health.